



---

UNIVERSIDAD CATÓLICA DE SALTA  
Licenciatura en Ciencia de Datos

---

# Trabajo Práctico N° 1

## Programación II

**Alumno:** Facundo Luna

**Profesor:** Ing. Mario Martínez

**Fecha:** 17 de agosto de 2026

---

# Índice

|                                                               |    |
|---------------------------------------------------------------|----|
| Aclaraciones previas                                          | 2  |
| 1. Lenguajes de máquina, de bajo nivel y de alto nivel        | 3  |
| 2. Compiladores e intérpretes                                 | 4  |
| 3. Operadores aritméticos y lógicos                           | 5  |
| 4. Concepto de algoritmo                                      | 6  |
| 5. Programa, lenguaje de programación y tarea del programador | 7  |
| 6. Investigación sobre un lenguaje de programación popular    | 8  |
| 7. Fases de la resolución de problemas                        | 10 |
| 8. Variables y constantes                                     | 11 |
| Bibliografía                                                  | 12 |

## Aclaraciones previas

El presente trabajo práctico aborda preguntas introductorias y conceptuales de la materia Programación II, algunas a modo diagnóstico, otras a modo de nivelación y otras específicas del contenido de la Unidad 1. Para su elaboración se consultó el material de estudio de la materia (y de Programación I) y otras fuentes de información complementarias, tales como sitios de internet y bibliografía especializada.

# 1. Lenguajes de máquina, de bajo nivel y de alto nivel

*Comente con sus palabras que entiende por el concepto y en qué se diferencian los lenguajes de máquina, bajo nivel y alto nivel. Incorpore ejemplos.*

Un lenguaje de programación es el medio a través del cual una persona le indica a una computadora qué tareas debe realizar. Estos lenguajes se clasifican según qué tan cerca están de lo que entiende directamente el hardware o de lo que entiende una persona (Joyanes Aguilar, 2008).

**Lenguaje de máquina.** Es el único lenguaje que la computadora ejecuta de forma directa, sin necesidad de traducción. Está compuesto por secuencias de ceros y unos (código binario) que representan instrucciones específicas del procesador, como mover datos entre registros o realizar una suma. Es totalmente dependiente del hardware: un programa escrito en lenguaje de máquina para un procesador Intel no funciona en un procesador con arquitectura distinta. Por ejemplo, una instrucción podría verse como 10110000 01100001.

**Lenguaje de bajo nivel (ensamblador).** Es una evolución del lenguaje de máquina que reemplaza las cadenas binarias por mnemotécnicos (abreviaturas en inglés) más fáciles de recordar para el programador, como MOV, ADD o JMP. Sigue siendo muy cercano al hardware —cada instrucción del ensamblador corresponde a una instrucción de máquina— por lo que también depende del procesador para el cual fue escrito. Requiere un programa llamado *ensamblador* para traducirlo a lenguaje de máquina.

**Lenguaje de alto nivel.** Se aleja del hardware y se acerca al lenguaje natural humano (generalmente en inglés), utilizando palabras e instrucciones que expresan directamente el problema a resolver, sin preocuparse por los detalles internos del procesador. Ejemplos son Python, Java, C++ o C#. Estos lenguajes son independientes de la máquina: un mismo programa puede ejecutarse en distintas arquitecturas siempre que exista un compilador o intérprete adecuado.

En síntesis, la diferencia central entre estos tres niveles radica en la distancia respecto del hardware: cuanto más bajo es el nivel, más eficiente y rápido es el código pero más difícil de escribir y menos portable; cuanto más alto es el nivel, más sencillo resulta programar y más portable es el código, aunque suele requerir un paso de traducción (compilación o interpretación) antes de ejecutarse (Cairó, 2005).

[↑ Volver al índice](#)

## 2. Compiladores e intérpretes

*También con sus palabras, que entiende y diferencias entre Compiladores e Intérpretes. Incorpore ejemplos.*

Tanto el compilador como el intérprete son programas traductores: convierten el código fuente escrito por el programador en un lenguaje de alto nivel a instrucciones que la computadora puede ejecutar. La diferencia está en *cómo* y *cuándo* realizan esa traducción (Joyanes Aguilar, 2008).

**Compilador.** Traduce el programa completo de una sola vez, antes de ejecutarlo. Como resultado genera un archivo ejecutable independiente (código objeto o código máquina) que ya no necesita del código fuente ni del compilador para correr. Si el programa tiene errores, el compilador los informa todos juntos al finalizar la traducción, y el programa no se ejecuta hasta que se corrigen. Ejemplos de lenguajes compilados son C, C++ y Rust.

**Intérprete.** Traduce y ejecuta el programa línea por línea, en el momento en que corre. No genera un archivo ejecutable aparte: cada vez que se quiere correr el programa, se necesita tener el intérprete instalado. Si encuentra un error, detiene la ejecución en esa línea sin revisar el resto del código. Ejemplos de lenguajes interpretados son Python y JavaScript (en su forma tradicional).

En la práctica, muchos lenguajes modernos (como Python o Java) combinan ambas estrategias: primero compilan el código fuente a un código intermedio (bytecode) y luego ese bytecode es interpretado o compilado *just-in-time* por una máquina virtual, buscando un equilibrio entre portabilidad y velocidad de ejecución (Cairó, 2005).

La principal ventaja del compilador es la velocidad de ejecución del programa final, ya que no hay traducción de por medio en tiempo de ejecución. La principal ventaja del intérprete es la flexibilidad para probar y corregir código rápidamente, sin tener que recompilar todo el programa cada vez.

[↑ Volver al índice](#)

### 3. Operadores aritméticos y lógicos

*¿Qué operadores aritméticos y lógicos conoce?*

**Operadores aritméticos.** Se utilizan para realizar cálculos matemáticos entre valores numéricos (Joyanes Aguilar, 2008):

- Suma: +
- Resta: -
- Multiplicación: \*
- División: /
- División entera: DIV (en PSeInt) — descarta la parte decimal del resultado
- Módulo o resto de la división: MOD (en PSeInt) — devuelve el resto de una división entera
- Potenciación: ^

**Operadores lógicos.** Se utilizan para combinar o evaluar expresiones booleanas (verdadero o falso), típicamente dentro de condiciones (**Si**) o bucles (**Mientras**) (Cairó, 2005):

- Y (AND): devuelve verdadero solo si ambas condiciones son verdaderas.
- O (OR): devuelve verdadero si al menos una de las condiciones es verdadera.
- NO (NOT): invierte el valor lógico de una condición (de verdadero a falso, o viceversa).

Además, suelen mencionarse junto a estos los **operadores relacionales** (=, <>, >, <, >=, <=), que comparan valores y devuelven un resultado lógico (verdadero o falso), y que habitualmente se combinan con los operadores lógicos para construir condiciones más complejas.

[↑ Volver al índice](#)

## 4. Concepto de algoritmo

*¿Qué es un algoritmo?*

Un algoritmo es un conjunto finito, ordenado y no ambiguo de pasos o instrucciones que permite resolver un problema o realizar una tarea determinada (Cairó, 2005). Para que una secuencia de pasos sea considerada un algoritmo debe cumplir con ciertas características:

- **Finitud:** debe terminar después de un número finito de pasos.
- **Precisión (no ambigüedad):** cada paso debe estar definido con claridad, sin lugar a interpretaciones distintas.
- **Entrada:** puede recibir cero o más datos de entrada.
- **Salida:** debe producir al menos un resultado o salida.
- **Efectividad:** cada paso debe ser lo suficientemente básico como para poder realizarse, en principio, por una persona con lápiz y papel en un tiempo finito (Joyanes Aguilar, 2008).

Un ejemplo cotidiano de algoritmo es una receta de cocina: tiene un conjunto ordenado de pasos, ingredientes como entrada y un plato terminado como salida. En programación, un algoritmo es la base lógica sobre la cual luego se construye un programa, independientemente del lenguaje en el que se vaya a implementar.

[↑ Volver al índice](#)

## 5. Programa, lenguaje de programación y tarea del programador

*¿Qué es un programa? ¿Qué es un lenguaje de programación? ¿Y qué tarea es la que hace un programador?*

**Programa.** Es la implementación concreta de un algoritmo, escrita en un lenguaje de programación específico para que una computadora pueda ejecutarla. Mientras el algoritmo es la idea o el plan de solución (independiente del lenguaje), el programa es esa misma solución ya traducida a instrucciones que la máquina puede procesar (Joyanes Aguilar, 2008).

**Lenguaje de programación.** Es un conjunto de reglas, símbolos y palabras clave (sintaxis) que permite a una persona escribir instrucciones comprensibles tanto para sí misma como, luego de un proceso de traducción (compilación o interpretación), para la computadora. Ejemplos son Python, Java, C o PSeInt (este último pensado específicamente para el aprendizaje mediante pseudocódigo).

**Tarea del programador.** El programador es quien analiza un problema, diseña un algoritmo que lo resuelve y lo traduce (codifica) a un lenguaje de programación concreto, para luego probarlo, corregirlo y mantenerlo en el tiempo. Su trabajo no se limita a escribir código: incluye también entender el requerimiento, planificar la solución antes de programar y verificar que el resultado final cumpla con lo esperado (Cairó, 2005).

[↑ Volver al índice](#)



## 6. Investigación sobre un lenguaje de programación popular

*De los lenguajes de programación más populares en la actualidad (puede seleccionar uno del material de estudio u otro que Ud. considere), seleccione uno de ellos y desarrolle una pequeña investigación que incluya al menos:*

Se seleccionó el lenguaje **Python** para esta investigación, por ser en la actualidad el lenguaje de programación más utilizado según los principales índices de la industria.

### a) ¿Cómo surgió?

Python fue creado por el programador neerlandés **Guido van Rossum** y su primera versión se publicó en 1991. El nombre no hace referencia a la serpiente, sino al grupo de comedia británico *Monty Python's Flying Circus*, del cual van Rossum era admirador. El objetivo original del lenguaje era ofrecer una sintaxis simple y legible, que redujera el esfuerzo de escritura de código sin sacrificar potencia, tomando ideas de lenguajes anteriores como ABC, Modula-3 y C (Joyanes Aguilar, 2008). A lo largo de los años atravesó varias versiones mayores, siendo la más relevante el salto de Python 2 a Python 3 (2008), que introdujo cambios profundos en el lenguaje y que actualmente es el estándar utilizado.

### b) ¿Para qué se usa en la actualidad?

Hoy Python se utiliza en una enorme variedad de dominios: desarrollo web (con frameworks como Django o Flask), automatización de tareas (*scripting*), análisis y ciencia de datos, y muy especialmente en inteligencia artificial y aprendizaje automático, gracias a bibliotecas como TensorFlow, PyTorch y scikit-learn. Según el índice TIOBE, a comienzos de 2026 Python alcanzó una participación cercana al 22 %, la más alta registrada por cualquier lenguaje en la historia del índice, impulsada en gran parte por su rol central en el desarrollo de IA (TIOBE Software, 2026). También se destaca en el ámbito educativo, siendo frecuentemente el primer lenguaje que se enseña por su sintaxis simple.

### c) ¿Por qué es tan popular ese lenguaje?

Varios factores explican su popularidad actual. En primer lugar, se volvió el lenguaje de referencia para la inteligencia artificial: los frameworks dominantes en machine learning (TensorFlow, PyTorch, scikit-learn) están escritos en o para Python, y es el lenguaje más utilizado en los repositorios de GitHub relacionados con IA (TIOBE Software, 2026). En segundo lugar, su sintaxis clara y cercana al lenguaje natural reduce la curva de

aprendizaje para principiantes. En tercer lugar, cuenta con una enorme comunidad y un ecosistema de bibliotecas para casi cualquier tarea, lo que evita «reinventar la rueda». Por último, la encuesta de desarrolladores de Stack Overflow de 2025 registró para Python el mayor incremento interanual de todos los lenguajes relevados, lo que confirma una tendencia de crecimiento sostenido y no un pico puntual (Stack Overflow, 2025).

#### d) ¿Cómo lo clasificaría según su nivel de abstracción del procesador?

Python es un lenguaje de **alto nivel**. Su sintaxis está diseñada para ser comprendida por personas, no por el procesador, y no requiere que el programador gestione manualmente recursos de bajo nivel como direcciones de memoria o registros del CPU. Además, es un lenguaje **interpretado** (más precisamente, compilado a bytecode y luego ejecutado por una máquina virtual, la CPython VM), lo que aumenta su nivel de abstracción respecto de lenguajes compilados de más bajo nivel como C. Esta combinación —alto nivel más interpretación— es justamente lo que lo hace más lento en tiempo de ejecución que lenguajes como C o Rust, pero mucho más rápido y sencillo de escribir y depurar (Cairó, 2005).

[↑ Volver al índice](#)

## 7. Fases de la resolución de problemas

*¿Qué fases en la «resolución de problemas» conoce? Menciónelas y explique la importancia de la Fase IV.*

Siguiendo el enfoque clásico de resolución de problemas aplicado a la programación —basado originalmente en el método de George Polya y adaptado por los principales autores de metodología de la programación—, se distinguen habitualmente cuatro fases (Cairó, 2005):

1. **Análisis del problema:** consiste en comprender exactamente qué se pide resolver, identificando los datos de entrada disponibles, los resultados de salida esperados y las restricciones o reglas del problema.
2. **Diseño del algoritmo:** una vez comprendido el problema, se elabora la solución en un nivel abstracto (independiente del lenguaje), mediante pseudocódigo o diagramas de flujo, definiendo la secuencia lógica de pasos que resuelve el problema.
3. **Codificación (o implementación):** el algoritmo diseñado se traduce a un lenguaje de programación concreto, respetando su sintaxis, para obtener un programa ejecutable por la computadora.
4. **Prueba y depuración (*testing y debugging*):** se ejecuta el programa con distintos conjuntos de datos —incluyendo casos límite— para verificar que produce los resultados esperados, y se corrigen los errores (*bugs*) que se detecten.

**Importancia de la Fase IV.** La fase de prueba y depuración es fundamental porque es la que garantiza que el programa efectivamente resuelve el problema para el cual fue diseñado, y no solo que «compila» o «corre sin errores visibles». Un programa puede ejecutarse sin fallas técnicas y aun así producir resultados incorrectos frente a determinados datos de entrada (por ejemplo, valores límite, negativos o nulos que no fueron contemplados en el diseño). Sin esta fase, errores lógicos podrían pasar desapercibidos y llegar a un entorno de uso real, con consecuencias que van desde resultados erróneos hasta fallos críticos del sistema. Por eso se considera que probar un programa de forma exhaustiva —y no solo con el «caso feliz»— es tan importante como diseñarlo o codificarlo (Joyanes Aguilar, 2008).

[↑ Volver al índice](#)

## 8. Variables y constantes

*¿Qué es una variable y qué es una constante?*

**Variable.** Es un espacio de memoria identificado con un nombre (identificador), destinado a almacenar un dato cuyo valor puede cambiar durante la ejecución del programa. Toda variable tiene tres elementos: un nombre, un tipo de dato (entero, real, carácter, lógico, etc.) y un valor, que puede modificarse en distintos momentos del programa (Jonyanes Aguilar, 2008). Por ejemplo, en un algoritmo que calcule el promedio de notas de un alumno, la variable **Nota** irá cambiando de valor cada vez que se ingrese una nueva calificación por teclado.

**Constante.** Es también un espacio de memoria identificado con un nombre, pero cuyo valor se fija una única vez y permanece invariable durante toda la ejecución del programa (Cairó, 2005). Se utiliza típicamente para valores que no deben cambiar, como el número  $\pi$ , una tarifa fija o un límite de un sistema. Por ejemplo, en el Ejercicio 10 de este mismo trabajo práctico, el valor de la tarifa de la primera hora de estacionamiento (200 pesos) podría definirse como una constante, ya que no varía durante la ejecución del algoritmo.

En síntesis, la diferencia esencial entre ambas es la **mutabilidad**: la variable está pensada para almacenar datos que cambian a lo largo del programa, mientras que la constante representa un valor fijo que se declara una sola vez y se utiliza sin modificarse.

[↑ Volver al índice](#)

## Bibliografía

- Cairó, O. (2005). *Metodología de la Programación: Algoritmos, Diagramas de Flujo y Programas* (3.<sup>a</sup> ed.). Alfaomega.
- Joyanes Aguilar, L. (2008). *Fundamentos de Programación: Algoritmos, Estructura de Datos y Objetos* (4.<sup>a</sup> ed.). McGraw-Hill.
- Stack Overflow. (2025). 2025 Stack Overflow Developer Survey [Consultado en agosto de 2026].
- TIOBE Software. (2026). TIOBE Programming Community Index [Consultado en agosto de 2026].